

Nom : Samb

Prénom : Faty Malo

INE : N06275720202

Email : fatymalo.samb@unchk.edu.sn

**Projet : Conception et développement
d'une plateforme mobile intelligente
de participation citoyenne pour
l'amélioration des services publics**

Diagrammes

I. Diagrammes UML

Plantuml : <https://www.plantuml.com/plantuml/>

1. Diagramme de cas d'utilisation

```
@startuml
left to right direction
actor Citoyen
actor Administrateur
actor "Service technique" as Service
actor "Système IA" as IA
rectangle "Plateforme de participation citoyenne" {
    usecase "Créer un compte" as UC1
    usecase "Se connecter" as UC2
    usecase "Signaler un incident" as UC3
    usecase "Ajouter une photo" as UC4
    usecase "Ajouter une description" as UC5
    usecase "Choisir une catégorie" as UC6
    usecase "Définir une priorité" as UC7
    usecase "Géolocaliser automatiquement" as UC8
    usecase "Consulter mes signalements" as UC9
    usecase "Suivre le traitement" as UC10
    usecase "Recevoir des notifications" as UC11
    usecase "Voter pour un signalement" as UC12
    usecase "Consulter les incidents" as UA1
    usecase "Affecter un service" as UA2
    usecase "Modifier le statut" as UA3
    usecase "Gérer les catégories" as UA4
    usecase "Consulter les statistiques" as UA5
    usecase "Traiter un incident" as US1
    usecase "Détecter automatiquement la catégorie" as UI1
}
```

}

Citoyen --> UC1

Citoyen --> UC2

Citoyen --> UC3

Citoyen --> UC9

Citoyen --> UC10

Citoyen --> UC11

Citoyen --> UC12

UC3 .> UC4 : <<include>>

UC3 .> UC5 : <<include>>

UC3 .> UC6 : <<include>>

UC3 .> UC7 : <<include>>

UC3 .> UC8 : <<include>>

Administrateur --> UA1

Administrateur --> UA2

Administrateur --> UA3

Administrateur --> UA4

Administrateur --> UA5

Service --> US1

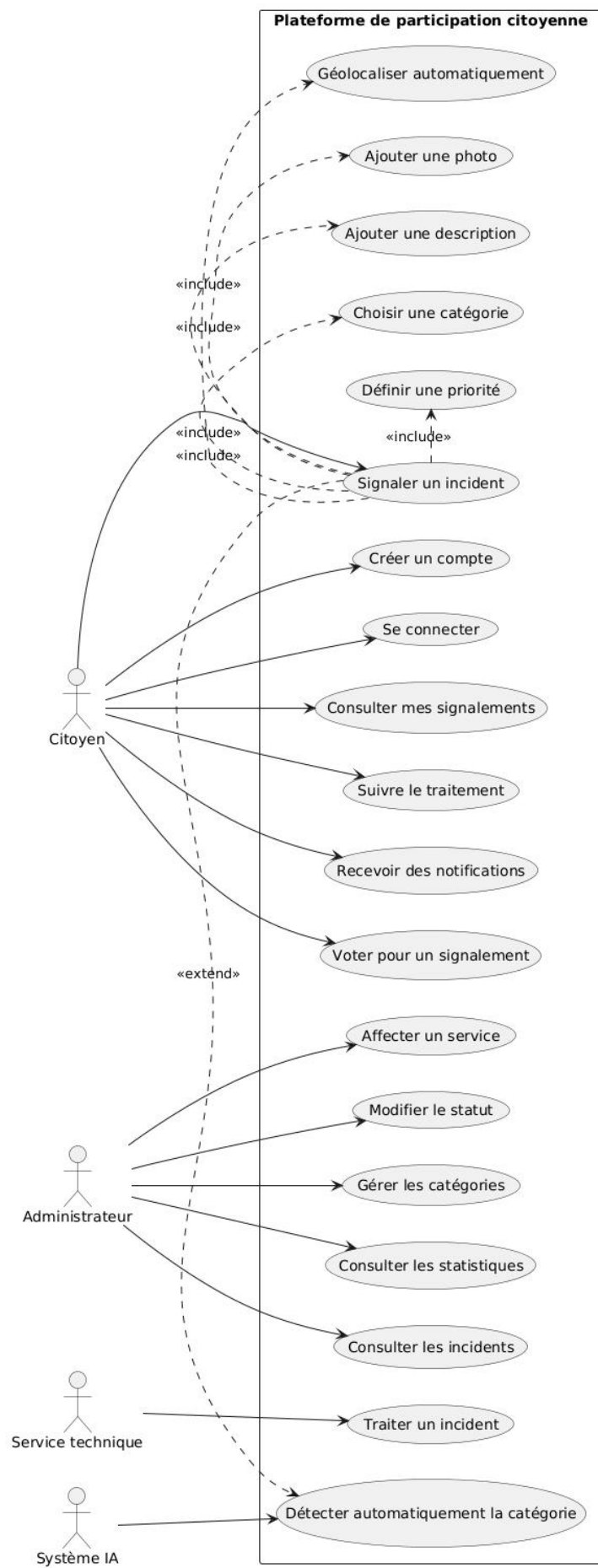
IA --> UI1

UC3 .> UI1 : <<extend>>

@enduml

❖ Acteurs

- Citoyen
- Administrateur
- Service technique
- Système IA (optionnel)



2. Diagramme de classes

@startuml

```
class Utilisateur{  
  -id:int  
  -nom:String  
  -email:String  
  -password:String  
  -role:String  
  +login()  
  +logout()  
}
```

```
class Citoyen
```

```
class Administrateur
```

```
Utilisateur <|-- Citoyen
```

```
Utilisateur <|-- Administrateur
```

```
class Incident{  
  -id:int  
  -description:String  
  -photo:String  
  -latitude:Double  
  -longitude:Double  
  -priorite:String  
  -statut:String  
  -dateCreation:Date  
}
```

```
class Categorie{  
  -id:int  
  -nom:String  
}
```

```
class Service{  
  -id:int  
  -nom:String  
}
```

```
class Notification{  
-id:int  
-message:String  
-date:Date  
}
```

```
class Vote{  
-id:int  
}
```

Citoyen "1" -- "0..*" Incident : crée

Categorie "1" -- "0..*" Incident

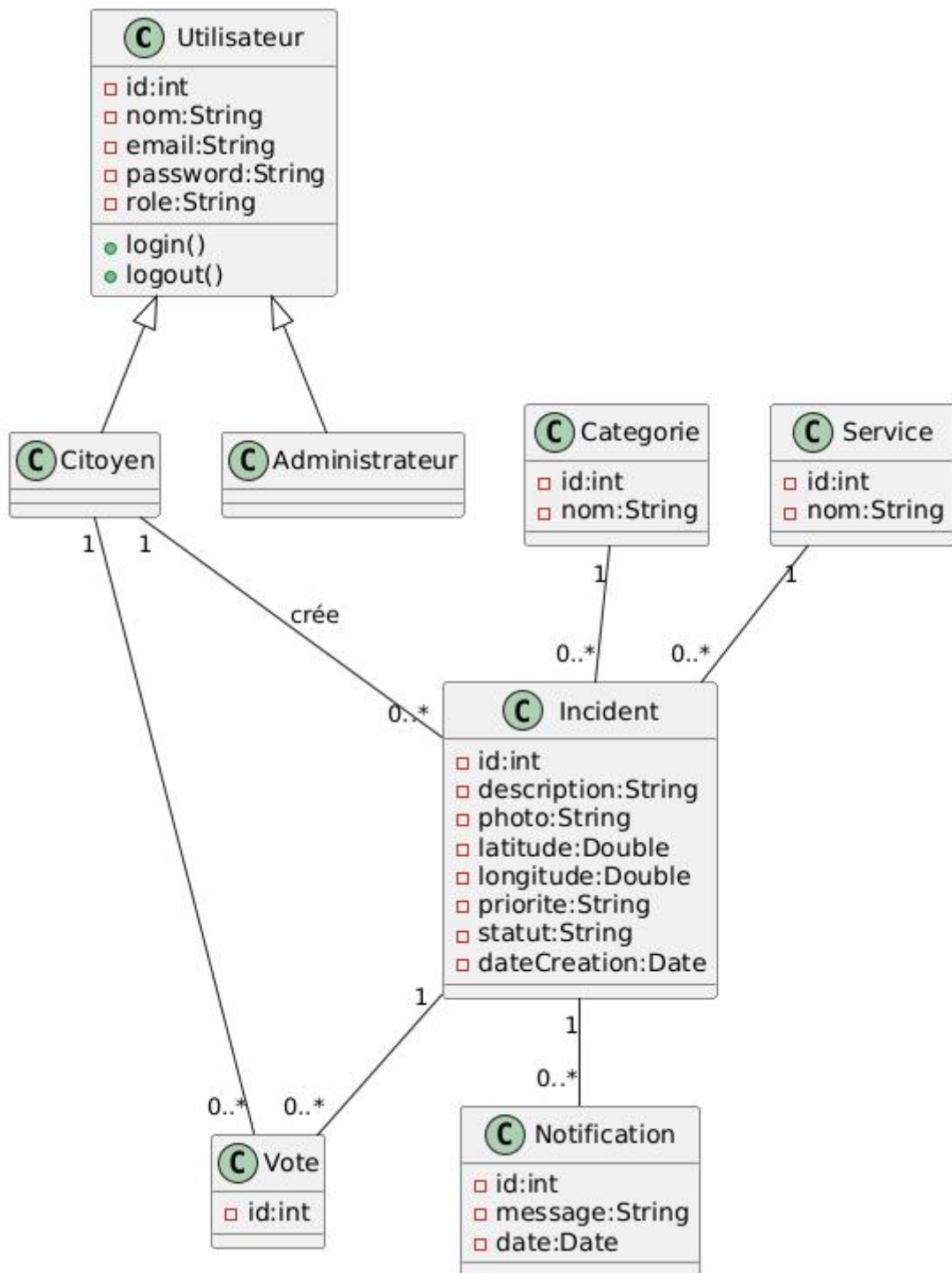
Service "1" -- "0..*" Incident

Incident "1" -- "0..*" Notification

Citoyen "1" -- "0..*" Vote

Incident "1" -- "0..*" Vote

@enduml



3. Diagramme de séquence (Signaler un incident)

@startuml

actor Citoyen

participant "Application Android" as App

participant "Google Maps" as Maps

participant "API REST" as API

database "Base de données" as DB

Citoyen -> App : Se connecter

Citoyen -> App : Créer un signalement

App -> Maps : Obtenir la position GPS

Maps --> App : Latitude/Longitude

Citoyen -> App : Ajouter photo et description

App -> API : Envoyer le signalement

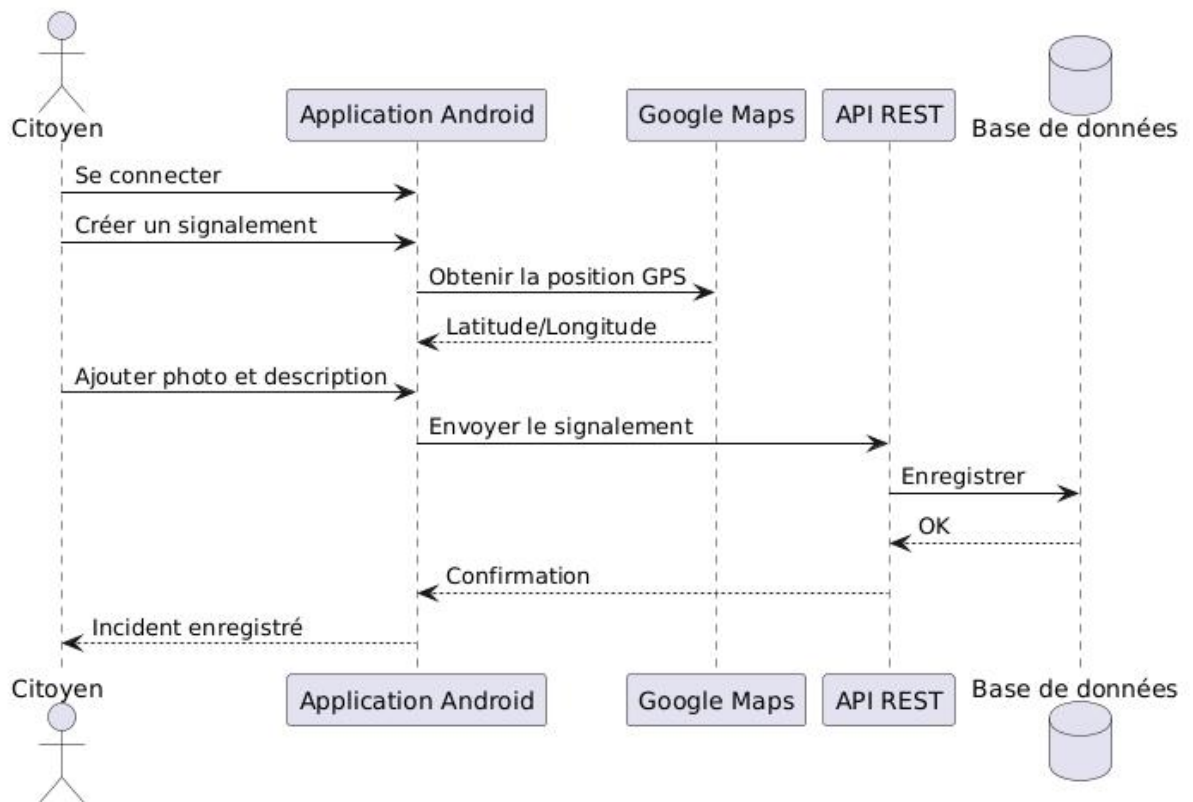
API -> DB : Enregistrer

DB --> API : OK

API --> App : Confirmation

App --> Citoyen : Incident enregistré

@enduml

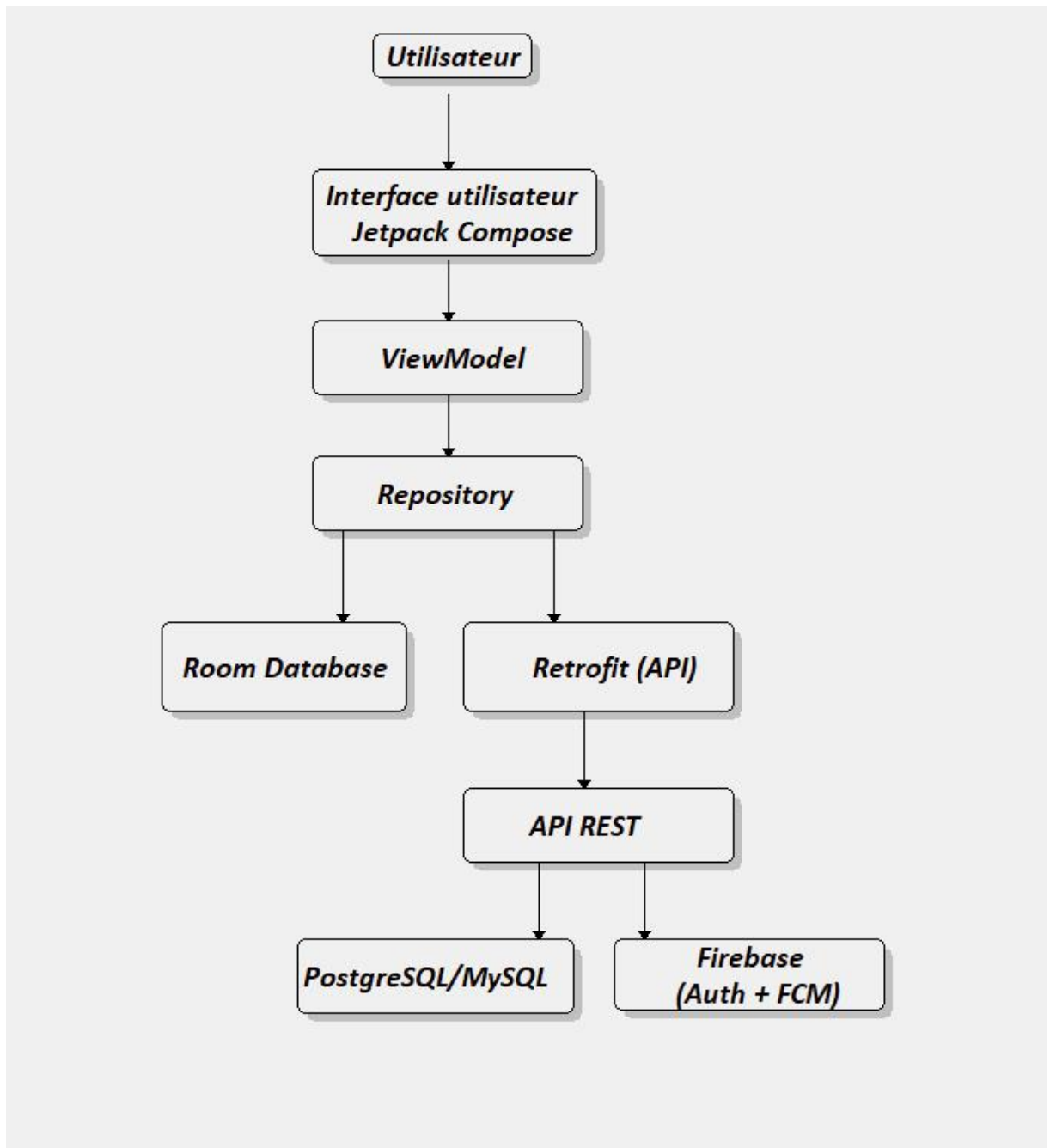


II. Conception de l'architecture logicielle

1. Architecture générale

L'application sera organisée selon l'architecture MVVM (Model–View–ViewModel), recommandée par Google pour les applications Android modernes. Cette architecture sépare les responsabilités entre l'interface utilisateur, la logique métier et l'accès aux données, ce qui facilite la maintenance, les tests et l'évolution du projet.

➤ Schéma d'architecture



2. Description des couches

➤ **Présentation (View)**

Responsable de :

- l'affichage des écrans ;
- la récupération des actions de l'utilisateur ;
- l'affichage des données.

Technologie :

- Jetpack Compose.

➤ **ViewModel**

Responsable de :

- la logique de présentation ;
- la communication avec le Repository ;
- la gestion de l'état de l'interface.

➤ **Repository**

Le Repository sert d'intermédiaire entre l'application et les sources de données.

Il :

- récupère les données du serveur ;
- lit les données locales (Room) ;
- décide quelle source utiliser.

➤ **Room Database**

Stocke localement :

- les incidents ;
- les catégories ;
- les notifications ;
- les données nécessaires au mode hors ligne.

➤ Retrofit

Permet :

- d'envoyer les requêtes HTTP ;
- de communiquer avec l'API REST ;
- de récupérer les données du serveur.

➤ API REST

L'API assure :

- l'authentification ;
- la gestion des incidents ;
- la gestion des catégories ;
- la gestion des services ;
- les statistiques ;
- les notifications.

➤ Firebase

Utilisé pour :

- l'authentification ;
- les notifications push (Firebase Cloud Messaging).

3. Flux d'un signalement

1. Le citoyen ouvre l'application.
2. Il se connecte.
3. Il remplit le formulaire de signalement.
4. L'application récupère automatiquement sa position GPS.
5. Il ajoute une photo.
6. Le ViewModel transmet les données au Repository.
7. Le Repository envoie les données via Retrofit.
8. L'API enregistre le signalement dans la base de données.
9. Une notification confirme l'enregistrement.
10. L'administrateur voit le nouveau signalement et peut le traiter.

4. Technologies utilisées

Couche	Technologie
Interface	Jetpack Compose
Architecture	MVVM
Réseau	Retrofit
Base locale	Room
Backend	Spring Boot, Laravel ou Node.js
Base serveur	PostgreSQL ou MySQL
Authentification	Firebase Authentication ou JWT
Notifications	Firebase Cloud Messaging
Cartographie	Google Maps SDK
IA	TensorFlow Lite

III. Maquettes de l'application

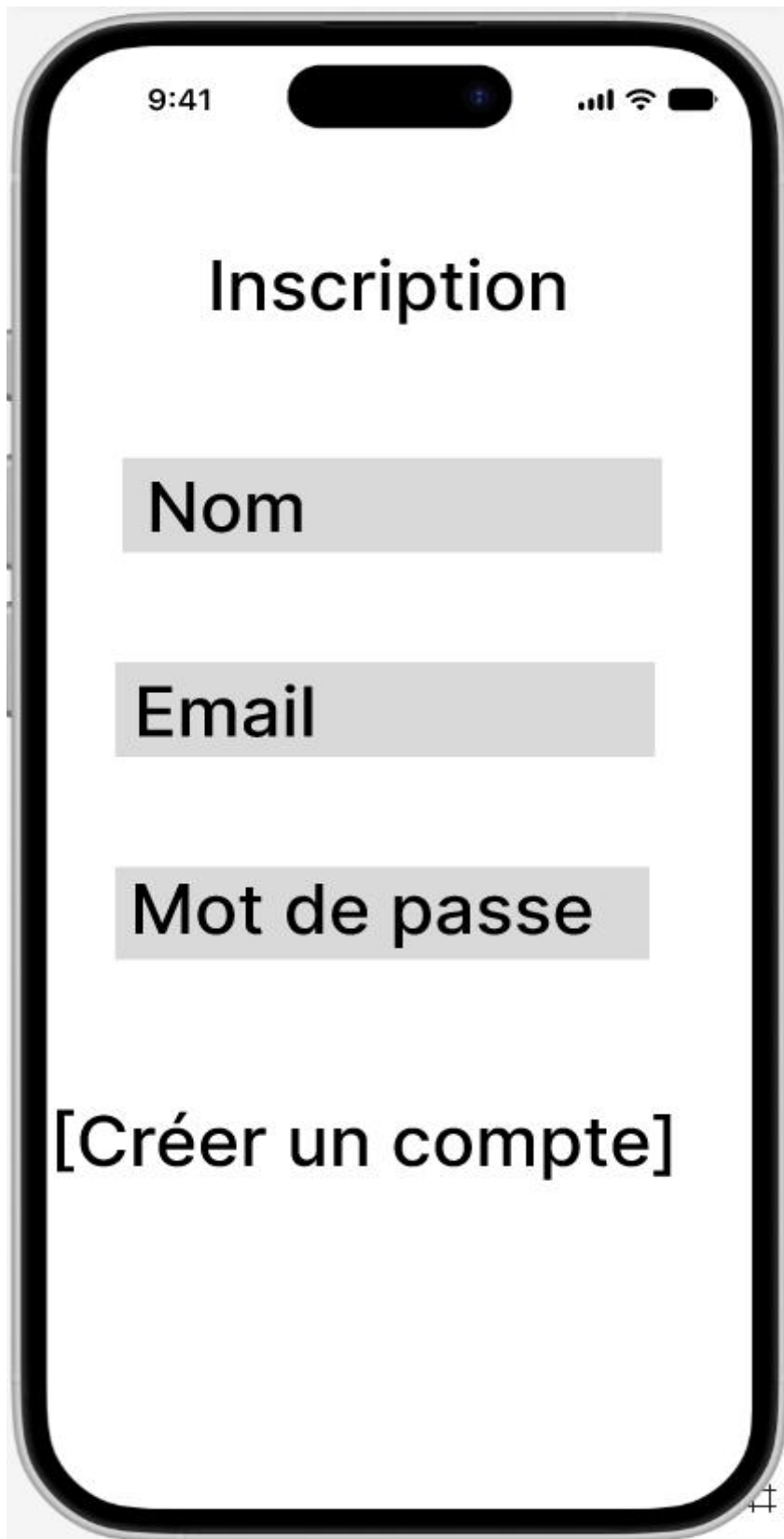
1. Écran Splash



2. Écran Connexion



3. Écran Inscription



9:41

Inscription

Nom

Email

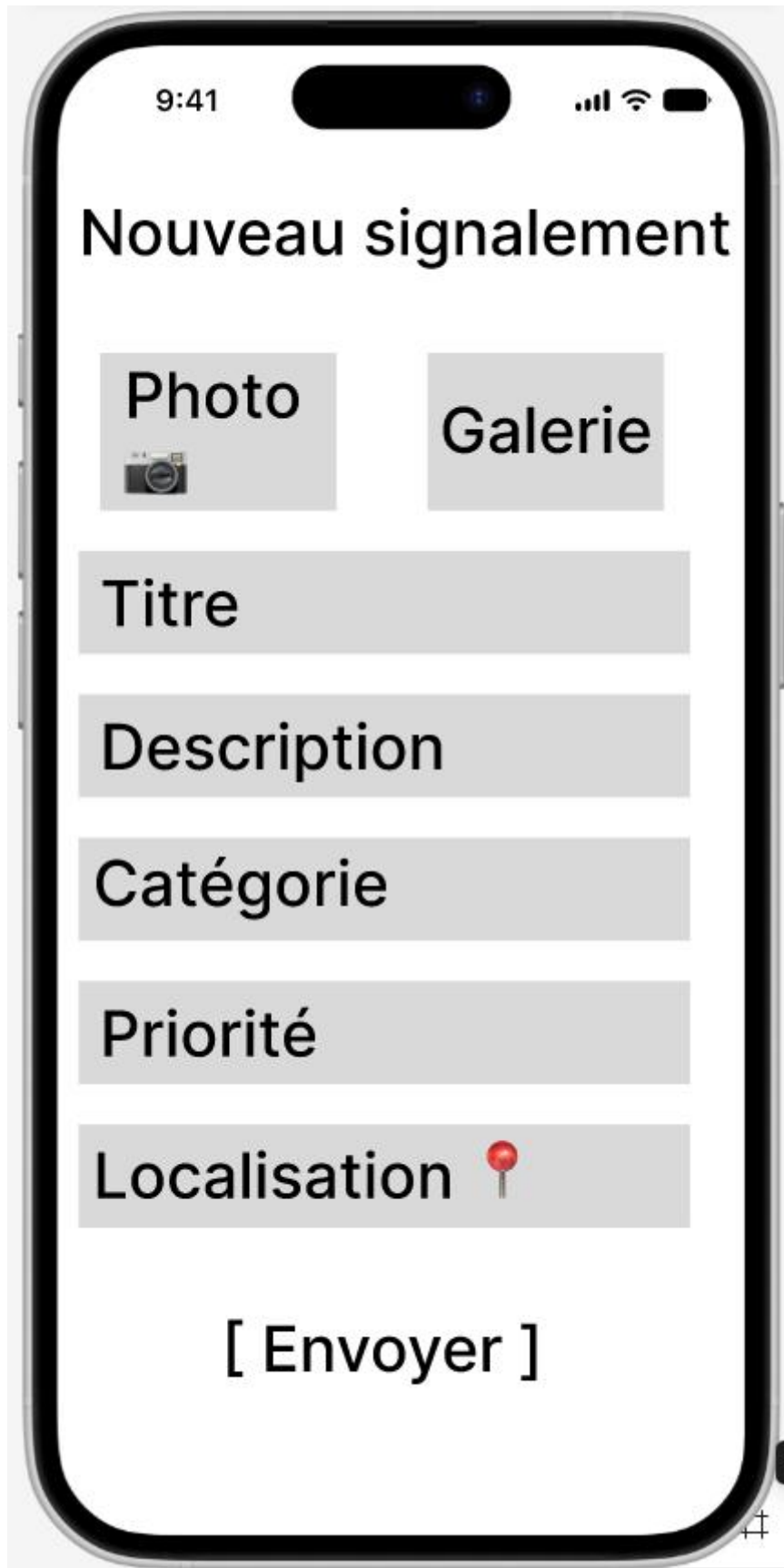
Mot de passe

[Créer un compte]

4. Écran Accueil



5. Écran Signalement



9:41

Nouveau signalement

Photo 

Galerie

Titre

Description

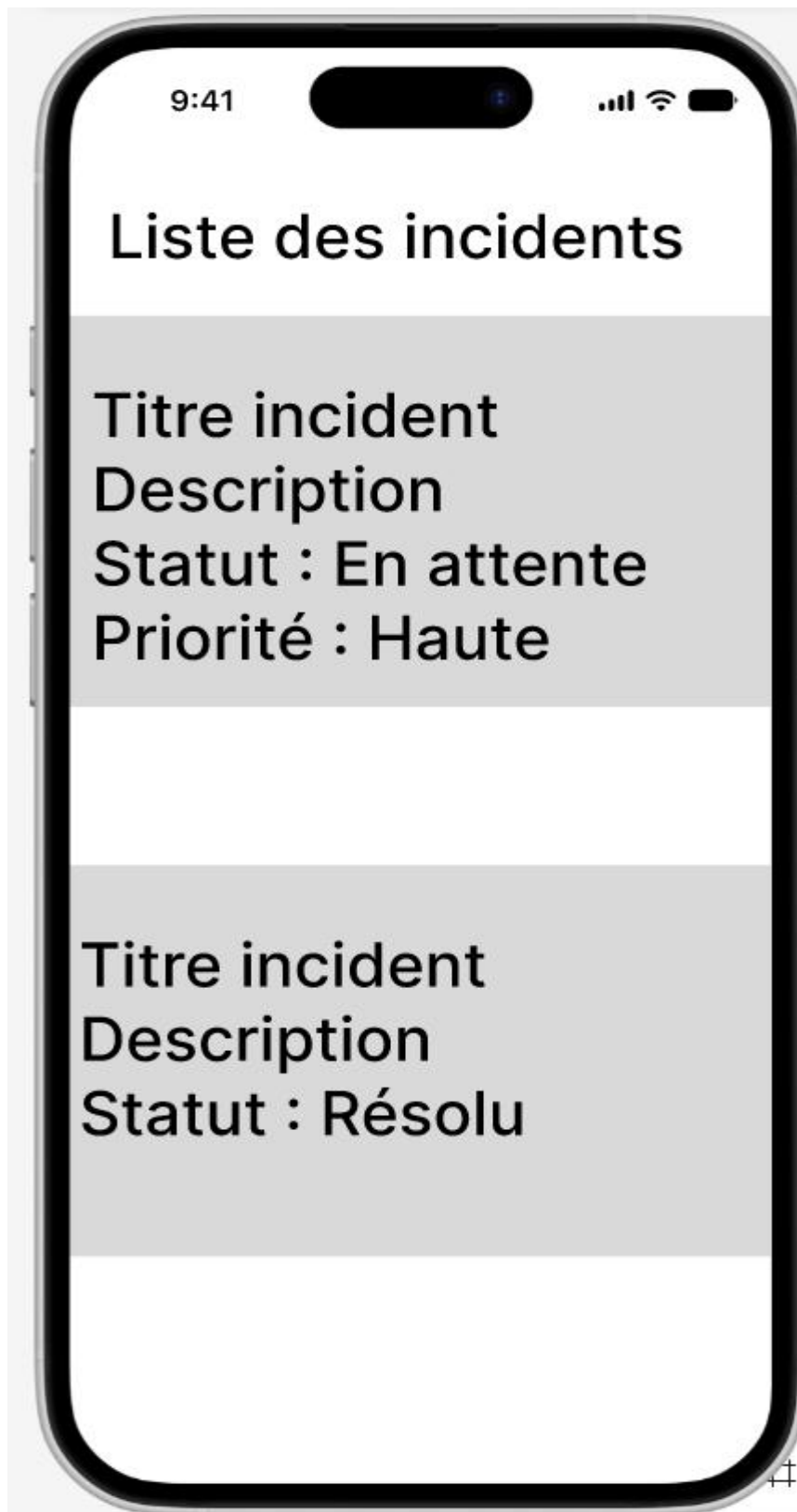
Catégorie

Priorité

Localisation 

[Envoyer]

6. Écran Liste des incidents



7. Écran Carte



8. Écran Profil

